Informe Trabajo Práctico Final

Nahuel Deppen

2025

Estructura de Datos y Algoritmos 1

Contents

1 INTRODUCCIÓN **2**

2 DISEÑO **2**

2.1 Interfaz básica 2

2.2 Archivos 3

3 Bibliografía **15**

1 INTRODUCCIÓN

Como trabajo práctico final de la materia, se requiere desarrollar un parser que permite transformar texto a una o varias estructuras de datos previamente definidas, un intérprete de funciones de listas que permite definir funciones de listas y listas, un evaluador de funciones de listas y un mecanismo que permite encontrar una secuencia de funciones a componer para transicionar de una lista a otra.

El siguiente informe tiene como propósito explicar y detallar las decisiones tomadas a la hora de realizar el proyecto.

2 DISEÑO

2.1 Interfaz básica

Al correr e iniciar el programa este estará en espera de la siguiente instruccíon. Para satisfacer el promt se deberá ingresar de a una sentencia a la vez, terminada en ';', estas pueden ser:

- **defl**:
  La sentencia *defl* crea una **Lista**, la cual posee nombre y cuerpo dada la instrucción: *'defl Nombre = [x,Y,z];'*.
  Donde *[x,Y,z]* representa una lista genérica de números naturales (se procesa el valor absoluto de los números ingresados).

- **deff**:
  La sentencia *deff* crea una **Funcion** a partir de **Funciones** predefinidas o **Funciones Primitivas** dadas las instruccines: *'deff NombreF = f1 f2 f3 ...;'* y *'deff NombreF = f1 <f2 f3> f5 ...;'*.

- **apply**:
  La sentencia *apply* aplica una función a listas del tipo **Lista** o *[x,Y,z]* dada las instrucciones: *'apply NombreF [x,Y,z];'* o *'apply NombreF Lista;'*.
  Esta no modifica la lista original, sino que imprime en pantalla el resultado de aplicar la función a la lista.

- **search**:
  La sentencia *search* crea una función de listas componiendo funciones ya definidas en el programa, tal que sea capaz de transformar pares de listas *Li1* a *Li2* dada la instrucción: *'search {L11, L12; L21, L22; ...; Ln1, Ln2};'*.

2.2 Archivos

El programa posee 5 archivos principales que controlan las instrucciones y sentencias ingresadas por el usuario: **parser.c**, **listas.c**, **funciones.c**, **search.c** y **main.c**; Acompañados por los archivos de estructuras de datos auxiliares: **tablahashOA.c**, **utils.c** y **colas.c**.

A continuación se presentara lo más destacable de los archivos principales:

2.2.1 parser

En el archivo **parser.c** se encuentran todas las funciones encargadas del manejo de la validez de las *instrucciones* ingresadas por el usuario. Como norma general, si en estas se determina que la linea ingresada no es válida imprimen un mensaje de **Error** seguido de una descripción del posible problema y retornan 0.
A destacar tenemos:

```c

int tokenizar(char linea[], char* tokens[]){
    int numTokens = 0;
    linea[strcspn(linea, "\n")] = '\0';
    int largoLinea= strlen(linea);

    for(int i = 0; i<largoLinea; i++){
        //ignora espacios
        if(isspace(linea[i]) || linea[i]=='\n')
        continue;
        if(isalnum(linea[i])){
            char secuencia[50];//tama o suficiente para almacenar una cadena o n mero entero
            int j = 0;
            while(i<largoLinea && isalnum(linea[i])){
                secuencia[j++]= linea[i++];
            }
            secuencia[j]= '\0';

            tokens[numTokens]=malloc(sizeof(char)* (strlen(secuencia)+1));
            strcpy(tokens[numTokens],secuencia);

            numTokens++;
        }
        //no se usa elseif para evitar saltear caracteres
        //detecta caracteres especiales
```

```
        if(strchr(",;][{}<>=",linea[i])){
            tokens[numTokens]= malloc(sizeof(char)*2);
            tokens[numTokens][0]= linea[i];
            tokens[numTokens][1]= '\0';

            numTokens++;
        }

    }
    tokens[numTokens]=NULL;

    return numTokens;
}
```

Listing 1: Definición de la función tokenizar

Esta se encarga de procesar todos los caracteres y strings importantes, creando una copia física de los mismos en *char* tokens[]* y retornando el número de caracteres procesado.

2.2.2 listas

En **listas.c** se encuentran todas las funciones que manejan:
La estructura de datos auxiliar **GLista**:

```
/**
 * Lista general doblemente enlazada con puntero al inicio y al
    final
 */
struct _GLista{
  Nodo izq;
  Nodo der;
};
typedef struct _GLista* GLista;
```

Listing 2: Definición de estructura GLista

y la estructura de datos **Lista**:

```
/*
 * estructura de la lista que evaluaran las funciones
 */
struct _List{
  char* nombre;
  GLista glista;
};
typedef struct _List* Lista;
```

Listing 3: Definición de estructura Lista

Esta es la estructura de las listas definidas por *'defl'*. Las cuales son creadas usando la función:

```c
/*
 * Crea una lista
 * dependiendo del operador crear una lista para DEFL o APPLY
 */
Lista lista_crear(char* tokens[], int numTokens,int tipoLista){

  Lista nuevaLista= malloc(sizeof(struct _List));
  assert(nuevaLista != NULL);
  nuevaLista->glista= glista_crear();

  switch(tipoLista){
    case(0):{//lista con nombre propio
      //lectura de datos para declaraciones DEFL
      nuevaLista->nombre= malloc(sizeof(char) * (strlen(tokens[1]) + 1));
      assert(nuevaLista->nombre != NULL);
      strcpy(nuevaLista->nombre,tokens[1]);


      for(int i = 4; i < numTokens-2; i+=2){
          int* dato= malloc(sizeof(int));
          *dato= atoi(tokens[i]);
          glista_agregar_der(nuevaLista->glista,dato, (FuncionCopia) copiar_puntero_entero);
          free(dato);
      }
      break;
    }
    case(1):{//lista generica, no se guardara en la tabla de listas
      //lectura de datos para declaraciones APPLY sobre listas genericas: [z,X,y]
      nuevaLista->nombre= malloc(sizeof(char) * (strlen(tokens[0]) + 1));
      assert(nuevaLista->nombre != NULL);
      strcpy(nuevaLista->nombre,tokens[0]);

      for(int i = 3; i < numTokens-2; i+=2){
          int* dato= malloc(sizeof(int));
          *dato= atoi(tokens[i]);
```

```
            glista_agregar_der(nuevaLista->glista,dato, (FuncionCopia)
               copiar_puntero_entero);
            free(dato);
        }

      }

    }
    return nuevaLista;
}
```

Listing 4: Función que crea Listas

Donde el primer **case** de la función se encarga de procesar arrays de tokens provenientes de haber parseado una instrucción *'defl'*. Al ser listas que se van a guardar posteriormente, estas reciben un nombre propio(dado por el usuario).
Para el segundo **case** tenemos un array de tokens provenientes de haber parseado un declaración *'apply'*, donde se ingreso una lista generica del tipo *[x,Y,z]*. Estas solo reciben el nombre de "apply" (tokens[0]) ya que luego de evaluarlas son eliminadas.

Se optó por mantener el manejo de **GLista** y **Lista** en el mismo archivo ya que muchas de las estructuras utilizadas en el programa que están definidas sobre la estructura **GLista**, son utilizadas para manejar **Listas**.

2.2.3 funciones

En el archivo **funciones.c** se encuentra todas las funciones encargadas del procesamiento de la sentencia *'deff'* y la aplicación de funciones de listas declaradas en *'apply'*.
Las estructuras de datos utilizadas para estas instrucciones son:

```
/**
 * tipo de funciones
 */
typedef enum {
    FUNC_OI,FUNC_OD,FUNC_SI,FUNC_SD,FUNC_DI,FUNC_DD,
    FUNC_USUARIO,
    FUNC_REPETICION
}TipoFuncion;
```

Listing 5: Definición del tipo de una función

Se eligió esta manera de representación para hacer más sencilla y metódicas las evaluaciones de listas más adelante.

```c
typedef void (*OperacionPrimitiva) (Lista);

typedef struct _funcionesPrimitivas{
    char* nombre;
    OperacionPrimitiva operacion;
}FuncionPrimitiva;
```

Listing 6: Definiciones usadas para las funciones primitivas

Se eligió este formato para poder definir y acceder de manera sencilla a las funciones primitivas (que en realidad son distintas operaciones sobre la estructura **GLista**) tal que:

```c
/*Funciones primitivas*/
const FuncionPrimitiva funcionesPrimitivas[6]=
{
    {"Oi",func_Oi},
    {"Od",func_Od},
    {"Si",func_Si},
    {"Sd",func_Sd},
    {"Di",func_Di},
    {"Dd",func_Dd}
};
```

Listing 7: Definición de las funciones primitivas

Luego, cada vez que se requiera hacer una operación primitiva sobre una lista simplemente se invoca la función:

```c
/**
 * Dada una Lista y un TipoFuncion  < FUNC_USUAIRO
 * evalua la lista segun el TipoFuncion
 */
void evaluar_primitiva(Lista lista, TipoFuncion t){
    funcionesPrimitivas[t].operacion (lista);
}
```

Listing 8: Función encargada de aplicar una operación primitiva sobre una lista

Estas funciones poseen nombres, con el objetivo de almacenarlas en una estructura:

```c
/**
 * Almacena nombres de funciones
 */
struct _nombres{
    int capacidad;
    char** arrNombres;
    int numNombres;
```

```
};
typedef struct _nombres* NombresFunciones;
```

Listing 9: Definición de la estructura NombresFunciones

En esta estructara se almacena el nombre de las funcione primitivas por default, ademas se almacenaran los nombres de todas las funciones que defina el usuario para usarlos más adelante en el metodo *'search'*.

Tenemos también la principal estructura usada por **funciones.c**:

```
struct _Funcion{
    //nombre de la funcion
    char* nombre;
    //lista de instrucciones
    Instrucciones cuerpo;
};
typedef struct _Funcion* Funcion;
```

Listing 10: Definición de la estructura Funcion

Como se puede observar, esta posee un nombre (definido por el usuario) y un **cuerpo de instrucciones**. Definido por:

```
// lista de instrucciones
typedef GLista Instrucciones;

struct _Instruccion{
    TipoFuncion funcionTipo;// en el caso que funcionTipo sea menor
       a 6, sabemos que es una primitiva
    union {
        // nombre de una funcion definida por el usuario
        char* nombre;
        //lista de instrucciones a repetir, donde cada dato es una
           instruccion
        Instrucciones listaRepeticion;
    };

};
typedef struct _Instruccion* Instruccion;
```

Listing 11: Definición de la estructura Instruccion

Podemos notar que una **Instruccion** se define con una *union*. Gracias a esto una **Instruccion** es capaz de almacenar cualquier tipo de función que usa el programa. Teniendo como caso base una **FuncionPrimitiva** (en el caso que funcionTipo sea menor a 6), en otro caso

el nombre de una función definida por el usuario (char*nombre) y una lista de instrucciones, para los casos recursivos. Sabemos cual de las dos ultimas guarda una instrucción ya que también guardamos su tipo en *funcionTipo*.
Gracias a este tipo de definiciones se pueden evaluar listas de manera recursiva usando:

```
/**
 * Aplica la instruccion dada sobre la lista
 */
void instruccion_evaluar(Instruccion inst,Lista lista,TablaHash
   tablaFunciones,int* numEvaluaciones);
/**
 * Dada una funci n de listas, una lista y una tabla de funciones:
 * Crea una copia de la lista dada y la evalua
 * En caso exitoso retorna la copia de la lista evaluada
 * en otro caso retorna NULL
 */
Lista funcion_evaluar(Funcion f, Lista lista,TablaHash
   tablaFunciones );
```

Listing 12: Funciones que evaluan Listas dadas una Instruccion y Funcion

funcion_evaluar se encarga de crear una copia de la **Lista** lista y evaluarla usando el *cuerpo* de la **Funcion** f a través de *instruccion_evaluar*. Notemos que se le pasa una **TablaHash**, que es donde se almacenan todas las funciones que define el usuario.
Así *instruccion_evaluar* evalua de manera recursiva una **Instruccion**, hasta llegar a los casos base, es decir las **funcionesPrimitivas**[] o hasta superar un máximo de evaluaciones sobre una lista.
Esto ultimo se hace por si el usuario ingresa una función que no está definida sobre los naturales, como *Pi = 0d 0d Sd <Sd Mi Sd Md> Di Dd Mi*(predecesor izquierdo), si terminaria en el caso de evaluarse *Pni= 0d Sd Si <Pi Dd Di Md> Dd Pi*(predecesor natual izquierdo)
Vale mencionar que la creación de una **Funcion** y una **Instruccion**, también se hace de manera recursiva.

Vale aclarar que, como las funciones se buscan por nombres en la **TablaHash**, estas no se pueden pisar. Por esta razon se crea un *dummy* cada vez que se busca una función. No sucede lo mismo para las listas.

2.2.4 search

Antesde explicar las estructuras y alguna de las funciones usadas en *search*, me gustaría explicar la idea detras del metodo usado para resolver el problema.

Basicamente, lo que se hace primero es crear pares de listas de *Origen* y *Destino*, se toman todas las listas de *Origen* y se crea una "lista" de listas(un array de **Listas**) que se asocia

a una composición de funciones (el caso base consiste en la composición "vacia" o "nula" asociada a las listas de *origen*)
Luego de esto, se crea una cola que se inicializa con una copia de las listas de *Origen* y se les aplican una por una las funciones almacenadas*[1] (se aplican a copias de las mismas por cada función existente),si cada una de estas no es adecuada se las encola. Luego se desencola una composición y el proceso se repite hasta encontrar la composición correcta o llegar a cierto limite de *profundidad*, ya que se esta haciendo un proceso de *BFS*, donde la cantidad de funciones aplicadas hasta el momento determina la *profundidad* que tiene el arbol de *composiciones*.

*[1] las funciones en el codigo se aplican en forma de **Instrucciones** que se crean de manera temporal.

En la imagen se ilustra el proceso del algoritmo suponiendo que se parte de una unica lista (la lista vacia), y se tienen definidas tres funciones (en este caso '0i', 'Si', 'Sd'). No se asume lista de destino y tampoco se desencola ninguna composición.

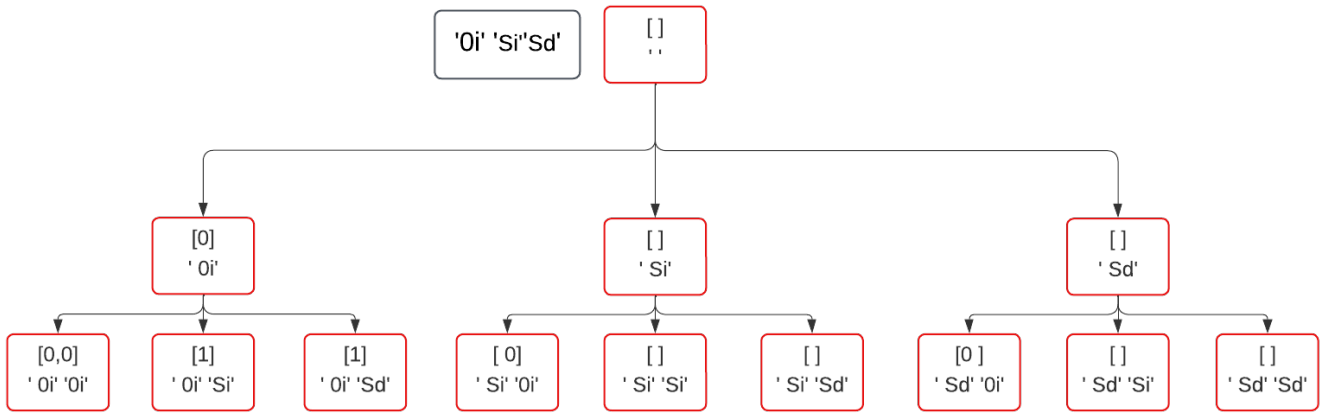

Figura 1: Ejemplo de aplicación del algoritmo comenzando con una lista vacia y teniendo tres funciones definidas

Podemos notar que el algoritmo tiene complejidad O($numeroDeFunciones^n$). Aunque en casos que existan funciones definidas que resulevan individualmente gran parte del proceso de la/s lista/s, no serán necesarios probar tantos casos.

Para este proceso entonces, tenemos las siguientes estructuras de datos:

```

/**
```

```
 * Estructura que guarda una lista de listas y el nombre de las funciones
 * que se usaron para obtenerlas(en orden)
 */
struct _Composicion{

    //Como solo se aceptan 20 pares, significa que a lo sumo habra 20 listasOrigen
    //es decir que a lo sumo se evaluaran 20 listas por composicion
    //estas listas son copias de su composicion previa(copia de las originaes en el primer caso)
    Lista listas[20];
    int numListas;
    //nombre de las funciones que se usaron para obtener las listas
    char** funcNombres;
    int numFunc;
    int profundidad ; //capacidad de la lista de nombres de funciones
};

typedef struct _Composicion* Composicion;
```

Listing 13: Definición de una Composición

Luego una **Composicion** tendra un array de **Listas** y otra de nombres (como no se sabe la cantidad de funciones que va a ingresar el usuario se define char *)

```

typedef struct _Pardelistas{
    //lista de la cual se parte para evaluar
    Lista listaOrigen;
    //lista objetivo a la cual se quiere llegar por medio de una composici n
    Lista listaDestino;
}ParDeListas;



//
struct _Pares{
    //se aceptaran 20 pares
    ParDeListas par[20];

    int numPar;
};

```

```
//array de pares de listas
typedef struct _Pares* ParesDeListas;
```

Listing 14: Definición de pares de listas

En el algoritmo de *search_aux* se usa *casoCompNull* para el caso de que la evaluación de una instrucción supere el número máximo de evaluaciones sobre una lista.

```

while(!colaVacia && !encontrada && inicioDeCola->numFunc < profundidadMax){
        comp= cola_desencolar(cola,(FuncionDestructora)composicion_destruir,(FuncionCopia)composicion_copiar);
        //en el siclo for  no se aumenta la profundidad de las composiciones (cant de nombres)
        for(int i=0; i < nombres->numNombres && !encontrada; i++){
            // como por cada nombre es una posible composicion de funciones e instruccion_evaluar
            //modifica la lista pasada, se crean copias de la comp inicial para ser evaluadas
            //y se encolan por si se usan mas tarde
            Instruccion inst= instruccion_simple_crear(nombres->arrNombres[i]);
            Composicion nuevaComp= nueva_composicion(inst,comp,tablaFunciones);
            Instruccion_destruir(inst);
            if(nuevaComp ==NULL){
                casoCompNull=1;
                break;
            }
            encontrada= composicion_adecuada(nuevaComp,paresDelistas);

            if(!encontrada){
                cola=cola_encolar(cola,nuevaComp,(FuncionCopia)composicion_copiar);
                composicion_destruir(nuevaComp);
            }
            else{
                compAdecuada= nuevaComp;
            }

        }
        composicion_destruir(comp);
        if(casoCompNull){
            compAdecuada=NULL;
```

```
            break;
        }

        colaVacia= cola_es_vacia(cola);
        inicioDeCola= cola_inicio(cola);
    }
    if(!cola_es_vacia(cola)){
        cola_destruir(cola,(FuncionDestructora)composicion_destruir);
    }

    return compAdecuada;
```

Listing 15: Fragmento de search_aux

2.2.5 main

En el archivo **main** se controla el programa dependiendo de la sentencia dada y de la validez de los tokens procesados. Particularmente para el control de *apply* veamos por partes lo que sucede:

```

struct _Funcion fTemp= {tokens[1],NULL};
    Funcion f=tablahash_buscar(tablaFunciones,&fTemp);
    if(f != NULL){//si se determina que la funcion fue definida por el usuario
        if(apply == 1){//apply = 1 -> apply nombreFun nombreLista;
            struct _List lTemp= {tokens[2],NULL};
            //la busqueda es segura ya que parse_apply determina
            //si la lista a usar fue definida previamente
            Lista lista= tablahash_buscar(tablaListas, &lTemp);
            lista= funcion_evaluar(f,lista,tablaFunciones);//lista ahora es la copia de la lista original o NULL
            if(lista){
                lista_imprimir(lista);
                lista_destruir(lista);// no se conserva la lista a la que se le aplico la funci n
            }
        }
        else{//apply = 2 -> apply nombreFun [x,Y,z];
            //se crea una lista generica temporal con el nombre "apply"
            Lista listaGenerica= lista_crear(tokens, numTokens, 1);
            Lista listaGenericaP= funcion_evaluar(f,listaGenerica, tablaFunciones);
```

```
            if(listaGenericaP){
                lista_imprimir(listaGenericaP);
                lista_destruir(listaGenericaP);// no se conserva la lista a la que se le aplico la funci n
            }
            lista_destruir(listaGenerica);
        }

    }
```

Listing 16: Aplicación de apply para funciones definidas por el usuario

En este fragmento podemos ver que lo que sucede si se busca aplicar una función definida por el usuario. Los casos internos determinan si se usa una **Lista** ya almacenada o se crea una **Lista** generica.
Analogamente para funciones primitivas tenemos este fragmento:

```

else if(primitiva_tipo(tokens[1])<6){// si la funcion es una primitiva
    if(apply == 1){//apply = 1 -> apply nombreFun nombreLista;
        struct _List lTemp= {tokens[2],NULL};
        //la busqueda es segura ya que parse_apply determina
        //si la lista a usar fue definida previamente
        Lista lista= tablahash_buscar(tablaListas, &lTemp);
        Lista listaCopia=lista_copia(lista);
        evaluar_primitiva(lista,apply);
        lista_imprimir(listaCopia);
        lista_destruir(listaCopia);

    }
    else{//apply = 2 -> apply nombreFun [x,Y,z];
        //se crea una lista generica temporal con el nombre "apply"
        Lista listaGenerica= lista_crear(tokens, numTokens, 1);
        evaluar_primitiva(listaGenerica, primitiva_tipo(tokens[1]));
        lista_imprimir(listaGenerica);
        lista_destruir(listaGenerica);
    }
}
```

Listing 17: Aplicación de apply para funciones primitivas

2.2.6 tablahashOA

Como se puede ver, esta es una implementación *OpenAdresing* de las tablas hash usando *Double hashing*.

```
//tabla de funciones
TablaHash tablaFunciones= tablahash_crear(CAPACIDAD_INICIAL_TABLA, (
   FuncionCopia)funcion_copia,
    (FuncionComparadora)funcion_comparar, (FuncionDestructora)
       funcion_destuir,
    (FuncionHash)funcion_hashear, (FuncionHash)funcion_hashear2);
//tabla de listas
  TablaHash tablaListas= tablahash_crear(CAPACIDAD_INICIAL_TABLA,(
     FuncionCopia)lista_copia,(FuncionComparadora)lista_comparar,
    (FuncionDestructora)lista_destruir, (FuncionHash)lista_hashear,(
       FuncionHash)lista_hashear2);
```

Listing 18: tablas de hash de funciones y listas

Podemos ver que para cada una de las tablas definidas en **main.c** les corresponden sus respectivas funciones de hash.

3 Bibliografía

- Slides de la catedra
- Algoritmos BFS y DFS: https://www.youtube.com/watch?v=_Yf8tneauJ8
- HashMaps & Dictionaries, Explained Simply: https://www.youtube.com/watch?v=y11XNXi9dgs&t=
- The C programming Language
- Estrucuras de Datos con C y C++ 2Ed